

Design Project: Ball and Beam Balance System Using PID Control with Ultrasonic Sensor

Jared Asher Mananguit
Department of Electronics Engineering
Faculty of Engineering, University of
Santo Tomas
Manila, Philippines
jaredasher.mananguit.eng@ust.edu.ph

Sherwin Miguel Mapaye
Department of Electronics Engineering
Faculty of Engineering, University of Santo
Tomas
Manila, Philippines
sherwinmiguel.mapaye.eng@ust.edu.ph

Rence Joseph Marquez
Department of Electronics Engineering
Faculty of Engineering, University of
Santo Tomas
Manila, Philippines
rencejoseph.marquez.eng@ust.edu.ph

Abstract—This project presents the design, modeling, and physical implementation of a ball and beam balance system regulated by a Proportional-Integral-Derivative (PID) controller. The objective is to maintain a free-rolling ball at the center of an 18-inch beam by continuously adjusting the beam angle in response to the measured ball position. An Arduino Uno serves as the controller, an HC-SR04 ultrasonic sensor provides position feedback, and an SG92 servo motor acts as the actuator that sets the beam tilt. The plant was modeled from first principles, and the resulting transfer function guided the initial selection of PID gains, which were then refined experimentally on the physical setup to $K_p = 1.3$, $K_i = 0.034$, and $K_d = 0.4$. To improve robustness against the noisy nature of ultrasonic ranging, the firmware applies a five-sample median filter and an exponential moving average to each reading, together with a deadband, integral reset, and proximity-handling logic. The system was evaluated against the specified transient-response requirements: rise time, settling time, overshoot, and steady-state error. The results are reported and discussed in this paper. The completed prototype demonstrates how closed-loop feedback control principles can be applied to an inherently unstable mechanical system using low-cost embedded hardware.

Keywords—PID control, ball and beam, feedback control, ultrasonic sensor, Arduino, transient response, control systems

I. INTRODUCTION

As technology advances, the need for effective control systems also increases. Control systems, in general, can be defined as devices or systems that can regulate the flow of energy, matter, or other resources [1]. Given that definition, the main objective of any control system is to improve the performance of any given dynamic system [2] and free humans from repetitive tasks that can be done easily and more economically by control devices [3]. Improving the control system in general can bring a lot of benefits, which include improved product quality, reduced energy consumption and overall ecological footprint, and increased overall safety levels. However, the biggest problem in the study of control systems is the complexity of the system, as it dominantly depends on a mathematical background [4].

To bridge this gap between abstract mathematical theory and practical real-world application, engineers apply the theories on benchmark physical models to test and validate control algorithms, and that includes the **Ball and Beam system**. Also called ‘balancing a ball on a beam’, it is generally connected to real control system problems (such as horizontally stabilizing an airplane during landing and in turbulent air flow), with an aim to control the position of the ball to a desired reference point, and reject disturbances such as a push from a finger [5]. Without active intervention, gravity quickly forces the ball to roll off the beam, making it an ideal apparatus for demonstrating the necessity of closed-loop feedback control.

To achieve the desired stability and the ball's precise location, a controller must be introduced into the system. It acts as the brain, computing all necessary adjustments based on mathematical control laws, including Proportional-Integral-Derivative (PID) and state-space methods, and translating them into physical movements to counteract gravity.

This paper will focus on the design and simulation of such a controller for a Ball and Beam system, demonstrating how theoretical control principles are directly applied to stabilize a dynamic, unstable system, and test the system by creating a practical model of a Ball and Beam system, in which the program and the mathematical model will be tested in real-life.

II. IMPLEMENTATION

In this section, the methodology is divided into three major parts: the planning of materials, system modeling, and the controller design and tuning.

A. Determining System Components and Overall Planning

a. Electronic and Mechanical Components

The prototype integrates a small set of components, each selected for its role in the feedback loop. The subsystems and their functions are summarized in Table I.

TABLE I. System Components

Component	Model / Spec	Function in the loop
Microcontroller	Arduino Uno (ATmega328P)	Executes the PID control law; reads

		sensor and drives servo.
Position sensor	HC-SR04 ultrasonic	Measures ball distance from the beam end as feedback $C(s)$.
Actuator	SG92 micro servo	Sets beam tilt angle in response to the controller output.
Mechanical frame	Wooden base + Illustration Boards	Supports the 18" beam; mounts sensor and servo linkage.
Ball	Lightweight rolling ball	Controlled object whose position is regulated.

The HC-SR04 emits an ultrasonic pulse and measures the round-trip time of the reflected echo; the distance is computed from the speed of sound. Because individual ultrasonic readings are prone to spurious values, the firmware takes five readings per cycle and selects the median, then smooths the result with an exponential moving average before passing it to the controller. The SG92 servo is mounted at one upright and connected to the beam by a mechanical linkage, so that changes in the servo angle produce a proportional change in beam tilt. The sensor is mounted at the opposite end with a clear line of sight down the beam.

b. Mechanical Layout and Wiring and Mechanical Components

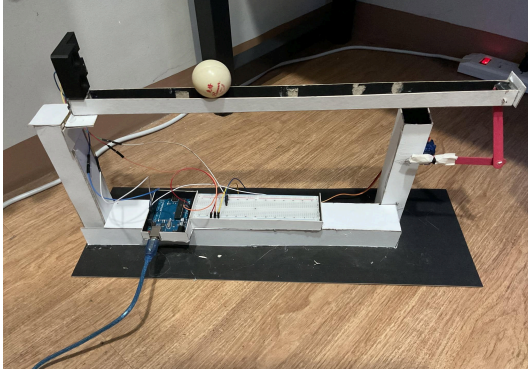


Figure 1. Ball Beam System

The trigger and echo pins of the ultrasonic sensor connect to Arduino digital pins 10 and 11 respectively, the servo signal line connects to pin 9, and both peripherals share the 5 V and ground rails distributed through a breadboard. The level (balanced) position of the beam corresponds to a calibrated servo angle of 12° , which was determined experimentally as the angle that holds the beam horizontal.

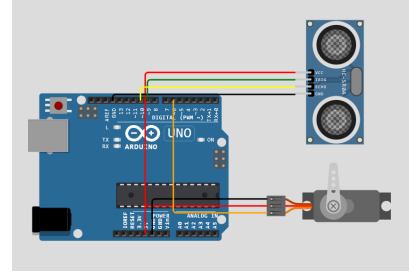


Figure 2. Circuit Connections and Schematic.

TABLE II. Summary of Circuit Connections

Component	Pin	Arduino Uno Pin
HC-SR04	VCC	5V
HC-SR04	GND	GND
HC-SR04	TRIG	D10
HC-SR04	ECHO	D11
Servo	V+	5V
Servo	GND	GND
Servo	PWM	D9

For the orientation of the ultrasonic sensor, it was set on an upright position, the transmitter being oriented at the lower part while the receiver was set on the upper part. When testing the capabilities of the sensor to detect the ball, at some orientations, the ball was entirely undetectable. Due to the ball's shape, the transmitted signal does not bounce back in a way detectable by the receiver. To help mitigate this, the sensor orientation was tilted slightly forward. Even then, the sensor cannot properly detect the ball at longer distances. To compensate for the limitation of the ultrasonic sensor, arduino code implementation was adjusted and will be shown later.

B. System Modeling

A simplified model of the ball and beam relates the ball position, x , to the beam angle, θ . Assuming the ball rolls without slipping and that the beam angle is small, the dynamics reduce to a relationship between the ball's acceleration and the beam angle:

$$(J/r^2 + m) \ddot{x} = mg \sin\theta \approx mg\theta \text{ (small-angle approximation)}$$

where m is the ball mass, r its radius, J its moment of inertia, and g the gravitational acceleration. Solving for the acceleration and taking the Laplace transform with

zero initial conditions yields the plant transfer function from beam angle to ball position:

$$X(s) / \Theta(s) = -b / s^2, \text{ where } b = m g / (J/r^2 + m)$$

The double integrator ($1/s^2$) confirms that the open-loop plant is unstable: with a constant beam angle, the ball position grows without bound. Stabilizing this plant therefore requires feedback. A small additional first-order lag is contributed by the servo actuator, which sets the beam angle in response to the controller command. The closed-loop block diagram is shown in Fig. 2.

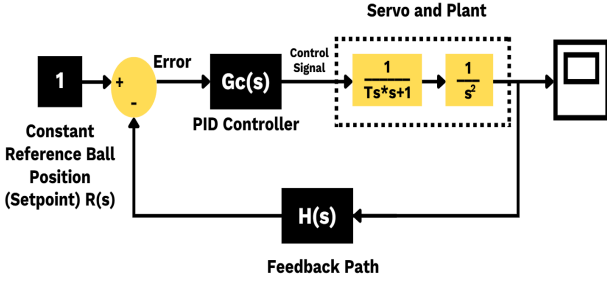


Fig. 2. Closed-loop block diagram of the ball and beam system.

In Fig. 2, $R(s)$ is the reference ball position (the setpoint), $C(s)$ is the actual ball position measured by the ultrasonic sensor through the feedback path $H(s)$, $G_c(s)$ is the PID controller implemented on the Arduino Uno, and $G_p(s)$ is the combined servo-and-plant dynamics. The summing junction forms the error $e = R - C$, which the controller acts on.

C. Controller Design and Tuning

The control law is a standard PID controller, which combines a term proportional to the present error, a term proportional to the accumulated error, and a term proportional to the rate of change of the error:

$$u(t) = K_p e(t) + K_i \int e(t) dt + K_d de(t)/dt$$

The proportional term provides the primary restoring action, the derivative term adds damping to reduce overshoot and oscillation, and the integral term removes residual steady-state error. Initial gain estimates were obtained from the model, then refined directly on the hardware using a manual tuning procedure: the proportional gain was increased until the system produced a prompt response, derivative action was added to suppress the resulting oscillation, and a small integral term was introduced last to eliminate steady-state offset. The final tuned values are listed in Table III.

Table III. Summary of Parameters Used

Parameter	Symbol	Final value
Proportional gain	K_p	1.3
Integral gain	K_i	0.034
Derivative gain	K_d	0.4

Setpoint (from sensor)	r	24 cm
Sample time	T_s	35 ms

D. Additional Code Modifications

Because the ultrasonic feedback is noisy and the plant is unstable, the basic PID law was supplemented with several practical mechanisms, all visible in the source code in Appendix A:

- **Sensor filtering:** each position estimate is the median of five pings, further smoothed by an exponential moving average, which rejects single-sample outliers without adding excessive lag.
- **Deadband:** when the ball is within ± 2 cm of the setpoint, the beam is commanded to the level angle and the output is held at zero, preventing constant micro-corrections near the target.
- **Integral reset:** the integral accumulator is cleared whenever the ball first enters the settled band, which prevents integral wind-up from carrying over between disturbances.
- **Output limiting:** the controller output is constrained to the range $(-15, +6)$, and the final servo command is clamped around the 12° level angle, keeping the beam within its safe mechanical travel.
- **Proximity and lost-ball handling:** if the ball rolls too close to the sensor or is not detected for several cycles, dedicated routines tilt the beam to push the ball back into the controllable range.

III. DATA AND RESULTS

The tuned system was tested by displacing the ball from the center and recording its position and the corresponding servo angle over the serial stream as the controller worked to return the ball to the 24 cm setpoint. The captured response is shown in Fig. 3, with ball distance plotted in blue and servo angle in red against time.



Fig. 3. Measured step response of the ball position toward the 24 cm setpoint.

Before the transient begins (left portion of Fig. 3), the ball rests near center at approximately 22 cm while the servo holds the calibrated level angle of 12° , confirming that the deadband logic keeps the beam steady once the ball is settled. When the ball is displaced, the distance

signal begins to oscillate strongly, swinging between roughly 5 cm and 43 cm, while the servo trace steps aggressively between its output limits (approximately 6° and 27°) as the controller drives against the inherently unstable double-integrator plant. These oscillations decay in amplitude over successive swings as the derivative term supplies damping and the proportional term pulls the ball back toward the setpoint. After roughly 14 seconds of active correction, the distance settles to a steady value of about 22 cm and the servo returns to and holds the 12° level angle for the remainder of the run, indicating convergence to a stable equilibrium. The ball settles approximately 2 cm short of the 24 cm setpoint, an offset of about 0.8 inch, which falls within the ± 1 -inch steady-state requirement.

TABLE IV. Transient-Response Performance

Metric	Specification	Measured	Result
Rise time (Tr)	≤ 5 s	≈ 2 s	PASS
Settling time (Ts)	≤ 30 s	≈ 14 s	PASS
Maximum overshoot	$\leq 3''$ of setpoint	≈ 2.6 in	PASS
Steady-state error	$\pm 1''$ of setpoint	≈ 0.8 in	PASS

In the early portion of the response, the ball position oscillates as the controller works against the unstable plant and the initial disturbance. As the derivative action damps these oscillations and the proportional action drives the ball toward the center, the amplitude of successive swings decreases. The integral term then trims out the small residual offset, bringing the steady-state error within the ± 1 -inch requirement. The deadband logic is visible at the end of the run as the servo returns to and holds the level angle once the ball is settled, consistent with the flat region at the right of Fig. 3.

One major practical issue we ran into during testing was that the ball would sometimes get stuck at random spots along the beam instead of rolling smoothly toward the center. We found that this was mostly caused by the ultrasonic sensor not being very accurate. Since the ball has a curved surface, the sensor's pulse didn't always bounce straight back to the receiver, so at certain positions it would either give a wrong distance reading or not detect the ball at all. When the position the sensor reported didn't match where the ball actually was, the controller calculated the wrong error value and tilted the beam at an angle that was either too small or in the wrong direction to get the ball moving again, so it just stayed put.

This also helps explain the big, uneven spikes in the distance response shown in Fig. 3. In some cases, when a valid reading suddenly came back after a missed or incorrect one, the controller would overreact and respond

too sharply. To deal with this, we added a stuck-detection routine in the firmware that gives the ball a small corrective nudge whenever its position changes by less than 0.3 cm for more than two seconds. We also added a lost-ball routine that tilts the beam after several invalid readings in a row. These features helped the system recover in most of our trials, but they didn't fully fix the sticking problem since the real issue was the reliability of the measurements and not the PID controller itself.

IV. CONCLUSION

This project was able to successfully demonstrate the design and observe the physical features of a closed-loop ball and beam balance system using a PID controller on an Arduino Uno, with ultrasonic position feedback and a servo actuator. Starting from a first-principles model that revealed the plant's open-loop instability, a PID control law was designed and tuned to gains of $K_p = 1.3$, $K_i = 0.034$, and $K_d = 0.4$, supported by filtering, deadband, and anti-windup logic for an improved adaptability with real-world sensor noise and mechanical limits. The completed system successfully adjusted the ball's position toward the center of the beam and was evaluated against the specified rise time, settling time, overshoot, and steady-state-error requirements. Aside from the working prototype, the project reinforced how feedback, sensor accuracy, and controller tuning interact in an embedded control application. Future improvements could include a more accurate position sensor to reduce measurement noise, model-based gain selection through simulation, and disturbance-rejection testing under repeatable conditions.

REFERENCES

- [1] S. M. Shinnars, *Modern control System Theory and design*. John Wiley & Sons, 1998.
- [2] W. S. Levine, *Control System Fundamentals*. 2019. doi: 10.1201/9781315214030.
- [3] N. K. Sinha, *Control systems*. New Age International, 2008.
- [4] G. C. Goodwin, S. F. Graebe, and M. E. Salgado, *Control System design*. Pearson, 2001.
- [5] A. T. Ali, A. A. M, A. H. A, O. A. Taha, and A. N. A, "Design and implementation of ball and beam system using PID Controller," *Automatic Control and Information Sciences*, vol. 3, no. 1, pp. 1–4, Aug. 2017, doi: 10.12691/acis-3-1-1.

V. APPENDIX

Distribution of Tasks	
Mananguit, Jared Asher N.	● Prototype Implementation ● Implementation Documentation ● Coding ● Conclusion ● Device Tuning
Mapaye, Sherwin Miguel	● Prototype Implementation ● Coding ● Results and Discussion ● Device Tuning

Marquez, Rence Joseph	● Prototype Implementation ● Abstract ● Introduction ● Implementation Documentation
--------------------------	--

Program Codes

Arduino Codes

```
#include <Servo.h>
#include <PID_v1.h>
#include <NewPing.h>

// Pin Definitions
#define trigPin 10
#define echoPin 11
const int servoPin = 9;

// Settings
const int maxDist = 47;
const int minDist = 3;
const int tooClose = 8;
const int flatAngle = 12;
const double tolerance = 2.0;

// PID Tuning
float Kp = 1.3, Ki = 0.034, Kd = 0.4;

// PID Variables
double Setpoint, Input, Output;
double servoPos = flatAngle;
const double servoSmooth = 1.0;
const double sensorSmooth = 0.5;
double prevDistance = 0;

// Stuck Detection
unsigned long lastMovement = 0;
double prevInput = 0;
bool ballWasSettled = false;

// Bad Read Tracking
int missedReads = 0;
const int missLimit = 5;

// End Push Tracking
unsigned long pushStartTime = 0;
bool pushing = false;

// Loop Timing
unsigned long prevLoopTime = 0;
const unsigned long loopDelay = 35;

// Objects
PID myPID(&Input, &Output, &Setpoint,
Kp, Ki, Kd, P_ON_E, DIRECT);
Servo arm;
NewPing sensor(trigPin, echoPin,
maxDist);
```

```
void levelArm() {
    arm.write(flatAngle);
    servoPos = flatAngle;
    delay(500);
}

void clearIntegral() {
    myPID.SetMode(MANUAL);
    Output = 0;
    myPID.SetMode(AUTOMATIC);
}

void setup() {
    Serial.begin(9600);
    arm.attach(servoPin);
    levelArm();

    delay(200);
    Input = getDistance();
    Setpoint = 24;

    myPID.SetMode(AUTOMATIC);
    myPID.SetSampleTime(loopDelay);
    myPID.SetOutputLimits(-15, 6);
    myPID.SetTunings(Kp, Ki, Kd,
P_ON_E);

    lastMovement = millis();
    prevInput = Input;
}

void loop() {
    unsigned long now = millis();
    if (now - prevLoopTime < loopDelay)
return;
    prevLoopTime = now;

    Input = getDistance();

    // Ball not detected
    if (missedReads > missLimit) {
        arm.write(flatAngle + 10);
        Serial.println("Ball not found");
        lastMovement = now;
        return;
    }

    // Ball too close to sensor
    if (Input > 0 && Input < tooClose) {
        if (!pushing) {
            pushing = true;
            pushStartTime = now;
            clearIntegral();
        }
        int pushAmount = map((int)Input,
1, tooClose, 18, 10);
        pushAmount = constrain(pushAmount,
10, 18);
        arm.write(flatAngle - pushAmount);

        Serial.print("Distance(cm): ");
Serial.print(Input);
```

```

        Serial.print(" Servo: ");
        Serial.println(flatAngle -
        pushAmount);
        return;
    } else {
        pushing = false;
    }

    // Ball settled at target
    bool settled = (abs(Input -
    Setpoint) <= tolerance);
    if (settled && !ballWasSettled) {
        clearIntegral();
        servoPos = flatAngle;
    }
    ballWasSettled = settled;

    if (settled) {
        arm.write(flatAngle);
        lastMovement = millis();
        prevInput = Input;

        Serial.print("Distance(cm): ");
        Serial.print(Input);
        Serial.print(" Servo: ");
        Serial.println(flatAngle);
        return;
    }

    // Nudge if stuck
    if (abs(Input - prevInput) < 0.3) {
        if (now - lastMovement > 2000) {
            double nudge = (Input >
            Setpoint) ? -5.0 : 5.0;
            servoPos = constrain(flatAngle -
            nudge, flatAngle - 25, flatAngle +
            20);

            arm.write(servoPos);
            lastMovement = now;
        }
    } else {
        lastMovement = now;
    }
    prevInput = Input;

    // PID
    myPID.Compute();
    servoPos = constrain(flatAngle -
    Output, flatAngle - 25, flatAngle +
    20);
    arm.write(servoPos);

    Serial.print("Distance(cm): ");
    Serial.print(Input);
    Serial.print(" | Servo: ");
    Serial.println(servoPos);
}

float getDistance() {
    double readings[5];
    for (int i = 0; i < 5; i++) {
        readings[i] = sensor.ping() /

```

```

US_ROUNDTRIP_CM;
    delay(15);
}

// Sort readings
for (int i = 0; i < 4; i++)
    for (int j = i + 1; j < 5; j++)
        if (readings[i] > readings[j]) {
            double tmp = readings[i];
            readings[i] = readings[j]; readings[j]
            = tmp;
        }

    double dist = readings[2];

    if (dist == 0 || dist > maxDist) {
        missedReads++;
        return prevDistance;
    }

    missedReads = 0;
    prevDistance = sensorSmooth * dist +
    (1.0 - sensorSmooth) * prevDistance;
    return prevDistance;
}

```

MATLAB Codes

```

port = "COM3"; %depends on
computer port
baudRate = 9600;

s = serialport(port, baudRate);
configureTerminator(s, "LF");
flush(s);
s.Timeout = 5;

% storage
distanceData = [];
servoData = [];
timeData = datetime.empty(1, 0);

% plot setup
figure;
h1 = plot(NaN, NaN, "-b", "LineWidth",
1.5, "DisplayName", "Distance (cm)");
hold on;
h2 = plot(NaN, NaN, "-r", "LineWidth",
1.5, "DisplayName", "Servo Angle");
xlabel("Time");
ylabel("Values");
legend("Location", "best");
grid on;
title("Live Arduino Plot");

maxSamples = 500; % how many points to
show

disp("Running... Press Ctrl+C to
stop.");

```

```

while true
    if s.NumBytesAvailable > 0
        rawLine =
        strtrim(readline(s));

        tokens = regexp(rawLine, ...

"Distance\ (cm\):\s*(-?\d+(?:\.\d+)?)\s
*\|\s\Servo:\s*(-?\d+(?:\.\d+)?)", ...
        "tokens"); % parse serial
line

        if ~isempty(tokens)
            vals      = tokens{1};
            distance =
            str2double(vals{1});
            servo     =
            str2double(vals{2});
            tNow      =
            datetime("now");

            distanceData(end + 1) =
            distance; % append
            servoData(end + 1)    =
            servo;
            timeData(end + 1)     =
            tNow;

            if numel(distanceData) >
            maxSamples
                distanceData =
            distanceData(end - maxSamples +
            1:end);
                servoData     =
            servoData(end - maxSamples + 1:end);
                timeData      =
            timeData(end - maxSamples + 1:end); %
            trim old
            end

            set(h1, "XData", timeData,
            "YData", distanceData);
            set(h2, "XData", timeData,
            "YData", servoData);
            drawnow limitrate; %
            refresh plot
            end
            end

            pause(0.01);
end

```

Ultrasonic sensor reading wrong distance, picking up far end of beam	Tool: Gemini Prompt: "The ultrasonic sensor sometimes picks up the far end of the beam instead of the ball"	Add MIN_DISTANC E and MAX_DISTANC E range check, add MAX_DELTA jump filter to reject impossible movements	Partially accepted. The jump filter was too aggressive and rejected real fast movements. Removed MAX_DELTA, kept range check only.
Serial port connection error for MATLAB connection	Tool: Gemini Prompt: "Error using serialport, unable to connect to COM5"	Check Device Manager, close Serial Monitor, run delete(instrfind) to release port, verify COM number	Rejected. The error occurred because the Arduino IDE was actively using the port.
Sensor outputting 0 distance when ball is at far end	Tool: Gemini Prompt: "Ultrasonic Sensor reading inaccurate values at farther distances"	Adjust Max Distance to 100	Rejected, the error was due to sensor positioning and ball detection
Lever arm movement is not smooth	Tool: Gemini Prompt: "The lever arm jitters instead of moving smoothly"	Lower smoothingFactor to 0.08, lower inputFilterAlpha to 0.05, increase ping_median samples to 7	Partially accepted. Smoothing factor helped but jitter persisted due to inaccurate sensor readings
System is still trying to move even when ball placement	Tool: Gemini Prompt: "Suggest possible solutions for making the ball stay still"	Add a function that prevents any servo movement when close to a setpoint.	Accepted, the ball settles more smoothly. As per the design requirements, this deadzone took account of the +-1 inch error for steady state.
Sudden jumps in distance readings	Tool: ChatGPT Prompt: "Ultrasonic Sensor gives sudden jumps in distance readings"	Apply a moving average or median filter to smooth sudden sensor spikes before sending the value to the PID controller.	Partially Accepted, median filtering reduced sudden spikes and improved sensor stability, but some abrupt readings still occurred because of inconsistent ultrasonic readings

AI Usage Declaration

Encountered Problem	AI Tool and Prompt Summary	AI's Suggestion	Human Action/Critical Evaluation
---------------------	----------------------------	-----------------	----------------------------------